Beating a greedy Catan bot with a better evaluation, not deeper search

Team bakarwadi · 2-player Catan hackathon, 17 October 2026 · submitted agent: `comboop1`

1 Problem, budget and what it rules out

We play 2-player Catan (no player trades) on catanatron against F (catanatron's `ValueFunctionPlayer`), O (the organisers' trained bot) and other teams. The only hidden information is the opponent's unplayed development cards. The clock is about 300 ms plus 12.5 ms of CPU per decision, about 1 s per game, in Python on one core.

Before designing anything we measured the primitives (`tools/profile_prims.py`). Copying a state and executing one action costs 0.10 ms, a value-function call 0.09 ms, and one random playout to the end of the game **11 ms**. A decision can therefore afford 60–100 simulated actions, and at most one random playout. This budget shaped every later decision: MCTS with rollouts and deep alpha-beta do not fit. Our own reruns agreed. catanatron's AlphaBeta at depth 2, unclocked, beat F in only 57.5 % of 40 games, and its same-turn variant won 8 % under the clock. We concluded that the lever is **the quality of a one-ply evaluation**, with only very shallow search.

2 Understanding the opponent first

F scores a state lexicographically: public VPs at weight $3{\cdot}10^{14}$, production at 10^{8}, everything else at most 10^{4}. In 60 F-vs-F games, winners got 0.03 Largest Army titles and 0.17 VP cards per game. Players bought 2.3 development cards per game and lost 17 cards per game to sevens. F cannot see what its objective omits: hidden VP cards, the value of knights before they are played, and hand-size risk. Two VP sources are left uncontested: Largest Army (2 VP) and VP cards. Our v5 bot holds Largest Army in about 89 % of its wins against F, plus about 1.5 hidden VP cards per won game.

3 Method

3.1 A value function in victory-point units

$$V(s) = S_{\text{me}}(s) - k\,S_{\text{opp}}(s) + w_A\,\text{clip}(K_{\text{me}} - K_{\text{opp}}, \pm 2) + \tfrac{1}{2} w_L\,\text{clip}(L_{\text{me}} - L_{\text{opp}}, \pm 2)$$

Here K counts knights played and L is road length. Each player's score S is a sum of hand-designed features, and **every weight is in VP units**. The terms are:

- VPs: my true VPs; the opponent's public VPs plus 0.2 per unknown development card.
- Production: pips per resource, robber-aware, worth less as the leader's VP rises.
- Resource diversity, port value, and expansion reach (best spots 0, 1 or 2 roads away).
- Hand terms: card count, discard risk above 7 cards, "ready to build" bonuses.
- Development cards in hand and played knights.

VP units make each weight a checkable claim, such as "a hand that can build a city now is worth 0.47 VP". They also make errors visible: any term that values a *state* above the +1 VP of the *action* it enables is wrong (§6). The opponent weight $k = 0.74 < 1$ means "race, don't block".

A **regime** $\varphi \in [0, 1]$ per player moves the weights from expanding to building:

$$\varphi = \max\Big(\text{clip}\big(\tfrac{\text{VP}-4}{4}\big),\, 1 - \min\big(1,\, \tfrac{\text{best open spot pips}}{10}\big)\Big)$$

As φ rises, wood and brick lose weight and wheat and ore gain it (×0.8 and ×1.2–1.3 at $\varphi = 1$). Expansion terms fade by $1 - 0.35\varphi$, and dev-card terms grow by $1 + 0.25\varphi$.

3.2 Decision procedure

We sample one world consistent with the observation, simulate *every* legal action on a copy, and pick the best V. Four refinements matter.

1. **Chance nodes in expectation.** Buying a development card is scored with the expectation over the unseen pile, not the one sampled draw: $\mathbb{E}[\text{dev}] = \sum_c \frac{n_c}{n}\, v(c) \approx 0.30$ VP. A robber steal counts as $+1$ card for us and -1 for them.

2. **Two-step follow-ups** for actions whose payoff is delayed. A trade, Year of Plenty or Monopoly is valued by the best build it enables; a Knight by its robber move; Road Building by its two roads.
3. **Turn plan.** In the main phase we take the top 3 first actions and play each greedily to the end of the turn. We switch from the one-ply choice only if a plan wins by more than 0.05 VP.
4. **Clock tiers.** The bot drops the turn plan below 0.45 s left, the follow-ups below 0.25 s, and all simulation below 0.10 s, where it uses fixed build priorities.

3.3 A learned opening

The four initial settlements fix most of a 2-player game's production, and the first decision has 300 ms of budget. We fit a logistic model of $P(\text{win} \mid \text{opening})$ on 16,000 self-play games (17 weights):

$$P(\text{win}) = \sigma\Big(\textstyle\sum_i w_i\,(x_i^{\text{me}} - x_i^{\text{opp}}) \pm w_{\text{seat}}\Big)$$

In those games, each settlement was the bot's own pick, or with probability 0.5 a random node among the ten free nodes with the most pips. The random picks show the model openings the bot would never choose.

The model reached a held-out AUC of 0.74 and log-loss 0.600, against 0.693 for seat alone. Gradient boosting did no better. Its largest weights favour balance over raw production:

- distinct resources +0.28;
- a 3:1 port +0.24;
- min(wheat, ore) pips +0.17;
- total pips +0.16.

At each pick we search the rest of the snake draft (P1, P2, P2, P1). The opponent's picks are predicted with our previous deterministic draft formula, and the four final settlements are scored with the model. The search stops at a CPU deadline.

4 Design decisions and why

- **Hand-built features, tuned by self-play, instead of learned end-to-end.** Published from-scratch RL agents for Catan lose even to random play, and a day is too short. A linear evaluation (Buro's Othello approach) is cheap to evaluate, which matters at 0.1 ms per call.
- **SPSA on win rate, against ourselves only.** Win rate is a noisy, non-differentiable objective, and SPSA needs just two evaluations per step, whatever the dimension. Weights live in log space, $\theta_k = \theta_k^0 e^{x_k}$. Each iteration plays $\theta^{\pm} = x \pm 0.25\Delta$, $\Delta \in \{\pm 1\}^d$, on 64 paired games, then updates $x \leftarrow x + a_t(2s - 1)\Delta/2$, where s is θ^+'s win rate. 93 iterations took about 12 minutes on 4 cores. **F was never an opponent, teacher or selection signal.** It was a held-out measuring stick only, so our F number is not overfitted.
- **Selection on paired head-to-heads.** In these, both bots play the same boards and dice from both seats; we required ≥ 300 games and about 2 SE before adopting a change.
- **Expectation at chance nodes.** Scoring on a single sample turns the decision into a coin flip. The turn plan proved it: scored on the sampled dev draw, it lost 0.460 against its parent; scored in expectation, it won 0.510–0.530.

5 Experiments

Every "vs F" number uses the official harness (`catan_kit.evaluate`, split over processes, CPU clock enforced, seats alternated, fixed seeds). The 95 % interval is $\pm 1.96\sqrt{p(1-p)/n}$. Head-to-head (h2h) numbers are in-process self-play on paired boards.

Version	Change	vs F	n	h2h vs parent
v1	value function + trade→build follow-up	0.635 ± 0.067	200	
v2	+ knight / road-building follow-up	0.683 ± 0.046	400	
v3	+ snake-draft opening lookahead	0.726 ± 0.028	1000	0.537 ± 0.056
v5	+ SPSA self-play weights (submission 1)	0.791 ± 0.025	1000	0.550 ± 0.049
combo	v5 + regime + turn plan	0.787 ± 0.025	1000	0.524 ± 0.018
op1	v5 + learned opening	0.786 ± 0.020	1600	0.543 ± 0.040
comboop1	**combo + learned opening**	**0.820 ± 0.024**	**1000**	**0.523 ± 0.031**

Table 1. Progression. In the "h2h vs parent" column, v5's parent is v3, combo's and op1's is v5, and comboop1's is combo. On the same 1000 seeds, comboop1 beat F 0.820 and combo 0.787; on op1's 1600 games, op1 0.786 and v5 0.761.

Other results.

- **Leaderboard (submission 1, v5):** vs F 0.779 ± 0.026 and vs O 0.626 ± 0.030, with 0 games over the clock. Our local F estimate was 0.787–0.791. [TODO: leaderboard numbers for comboop1.]
- **Official harness, comboop1:** 0.805 vs F (200 games), 0 games over the clock.
- **Robustness of v5 (h2h):** it beats catanatron AlphaBeta-2 0.883 (60 games, unclocked) and WeightedRandom 1.000.
- **A second strong benchmark:** our re-implementation of the organisers' "F + regime" bot beats F 0.570. v5 beats it 0.747. The difficulty ladder is consistent: F, then F+regime, then O.

Tried and not adopted	h2h vs v5	n
Pure learned linear evaluation (logistic regression on outcomes)	0.10–0.25	≥ 300
Opponent-roll expectimax at end of turn (11 dice sums, worst-case robber)	0.42–0.46	400
Convex "progress toward a build" hand features (vs v2)	0.417 ± 0.056	300
Endgame danger mode; robber lookahead	0.497–0.498	600
Second SPSA round; SPSA with regime switched on	0.503	600–1200
1v1 community heuristics (knight hold, bottleneck robber, …)	0.47–0.51	600 each
Opponent-adaptive Thompson-sampling bandit over 4 strategies	0.530	400

Table 2. Negative and neutral results. The adaptive bandit scored 0.805 vs F, 400 games, clocked. It was not adopted because it is not distinguishable from comboop1 at that sample size.

6 Insights

1. **A model that predicts well is not a model that decides well.** Logistic regression of the winner on our own features had 78 % held-out accuracy and log-loss 0.445. As a greedy policy it won only 10 % against the hand-tuned bot. The fitted weights show why. "Holding the cards for a city" earned +1.48 VP, more than building the city gains. The bot hoarded and never built, and played knights even got a negative weight because they are collinear with army lead. Refitting on after-states only lifted it to 0.19, so the fix is not more outcome regression. The fix is a signal at the decision level: rank the chosen after-state against its siblings, or bootstrap as in TD learning. Any learned critic used greedily has this problem.
2. **Where the same learning worked, and why.** The opening model is the same kind of regression, yet it gained 2.5–5 points against every opponent. All four settlements are compared at one point in time and the choice changes only static assets, so for this decision, predicting the outcome and choosing the action are the same task. In the mid-game they diverge.
3. **Evaluation beats depth when compute is the constraint.** A better one-ply evaluation took us from 50 % to 79 % against F at F's own CPU cost. Search on F's evaluation gave 57.5 %, and none of our three search extensions on top of v5 cleared 2 SE.

4. **Exploit what the opponent's objective cannot see.** Largest Army and hidden VP cards are VP sources F never contests, and they show up in almost all of v5's wins.
5. **Once self-play has tuned an evaluation, small structural changes get absorbed.** A second SPSA round, a regime and race weights each landed within ±2 SE of v5. Two changes of +1.7 SE each did not add up (0.508). New information, such as the learned opening, still helped.
6. **Measurement discipline.** SPSA's own θ^{+}-vs-θ^{-} score sits at 0.5 by construction, so progress must be measured against a frozen parent. At n = 400 the interval is ±4.4 points, and several "improvements" seen at n = 40–60 vanished at n = 300. Paired boards and a held-out opponent were necessary.

7 Limitations and next steps

- **One sampled world.** The bot ignores hidden-VP risk late in the game, and about 80 % of the clock goes unused.
- **Opening limits.** The opponent model in the opening is our own draft, the opening data is self-play only, and roads are ignored when an opening is scored.
- **Next steps.** TD(λ) on after-states, the principled fix for insight 1. Averaging over several sampled worlds. Joint SPSA of the evaluation with the turn plan switched on.

References

1. B. Collazo. *catanatron*: Settlers of Catan simulator and bots. github.com/bcollazo/catanatron (GPL-3.0).
2. J. C. Spall. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Trans. Automatic Control*, 37(3), 1992.
3. M. Buro. Improving heuristic mini-max search by supervised learning. *Artificial Intelligence*, 134, 2002.
4. G. Tesauro. Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38(3), 1995.
5. I. Szita, G. Chaslot, P. Spronck. Monte-Carlo tree search in Settlers of Catan. *Advances in Computer Games*, 2010.